PHP Variables

Variables are "containers" for storing information.

A variable can have a short name (like $x and $y) or a more descriptive name ($age, $carname, $total_volume).

Rules for PHP variables:

- A variable must start with the $ sign, followed by the name of the variable
- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive ($age and $AGE are two different variables)

Remember that PHP variable names are case-sensitive!

Create PHP Variables

In PHP, a variable starts with the $ sign, followed by the name of the variable:

ExampleGet your own PHP Server

Create two variables, named $x and $y:

```
$x = 5;

$y = "John";
```

In the example above, the variable $x will hold the value 5, and the variable $y will hold the value "John".

Note: When you assign a text value to a variable, put quotes around the value.

Note: Unlike other programming languages, PHP has no command for declaring a variable. It is created the moment you assign a value to it.

Output Variables

The PHP echo keyword is often used to output data to the screen.

The following example will show how to output some text and the value of a variable:

Example

```
$txt = "W3Schools.com";

echo "I love $txt!";
```

The following example will produce the same output as the example above:

Example

```
$txt = "W3Schools.com";

echo 'I love ' . $txt . '!';
```

The following example will output the sum of two variables:

Example

```
$x = 5;

$y = 4;
```

```
echo $x + $y;
```

Note: You will learn more about the echo keyword and how to output data in the PHP Echo/Print chapter.

PHP is a Loosely Typed Language

In the example above, notice that we did not have to tell PHP which data type the variable is.

PHP automatically associates a data type to the variable, depending on its value. Since the data types are not set in a strict sense, you can do things like adding a string to an integer without causing an error.

In PHP 7, type declarations were added. This gives an option to specify the data type expected when declaring a function, and by enabling the strict requirement, which will throw a "Fatal Error" on a type mismatch.

You will learn more about strict and non-strict requirements, and data type declarations in the PHP Functions chapter.

PHP Variables and Data Types

In PHP, the data type depends on the value of the variable.

Example

```
$x = 5;      // $x is an integer

$y = "John"; // $y is a string

echo $x;

echo $y;
```

PHP supports the following data types:

- **string** (text values)
- **int** (whole numbers)
- **float** (decimal numbers)
- **bool** (true or false)
- **array** (multiple values)
- **object** (stores data as objects)
- **null** (empty variable)
- **resource** (references external resources)
- **mixed** (any value)

Use var_dump() to Get the Data Type

To get the data type and the value of a variable, use the var_dump() function.

Example

The var_dump() function returns the data type and the value:

```
var_dump(5);

var_dump("John");

var_dump(3.14);
```

```
var_dump(true);

var_dump([2, 3, 56]);

var_dump(NULL);
```

Assign Multiple Values

You can assign the same value to multiple variables in one line:

Example

Here, all three variables get the value "Fruit":

```
$x = $y = $z = "Fruit";
```